Predicting TTC Subway Delays from Weather Data

JSC370 Final Project Report

Xiran Yin

2026-04-26

Table of contents

1 Introduction

Public transit is the circulatory system of any large city, and its reliability shapes how millions of people experience daily life. The Toronto Transit Commission (TTC) carries over one million riders per day across one of North America's largest subway networks, making service disruptions both economically costly and socially consequential. While many causes of delay — equipment failures, passenger incidents, security investigations — are largely random from a forecasting perspective, **weather** is one of the few systematic, externally observable, and forecastable variables that plausibly drives day-to-day delay patterns. Cold snaps freeze track switches; heavy snow buries third rails; extreme heat causes rail steel to expand and buckle; and heavy rain can flood at-grade sections of the line.

This project investigates whether and how daily weather conditions predict TTC subway delays, by integrating **12 years of meteorological observations (2014–2025)** with the City of Toronto's complete public record of subway delay incidents.

1.1 Central Research Question

> **Can daily weather conditions predict TTC subway delays, and if so, which weather features matter most — both for explaining day-to-day variation in delay duration and for forecasting severe delay days?**

This single question has two complementary halves: an *explanatory* half (which weather variables are associated with delays, and how?) and a *predictive* half (can we turn that association into a forecasting tool?). The two halves correspond to two formal hypotheses, each with an explicit null and alternative.

1.2 Hypotheses

H — Explanatory. *Adverse weather (precipitation, snowfall, and extreme temperature) is associated with longer total daily TTC delays, with potentially non-linear functional form.*

- **Null (H):** After controlling for temporal effects (month, day-of-week), the partial slopes of total daily delay on precipitation and snowfall are zero, and the smooth functions of precipitation and temperature are flat.
- **Alternative (H):** The partial slopes are positive (more precipitation → longer delays; more snowfall → longer delays), and the smooth functions exhibit non-linear structure (e.g., U-shaped temperature, saturating precipitation).

H is tested with OLS multiple regression for the linear slopes and a Generalized Additive Model (GAM) for the smooth non-linear terms.

H — Predictive. *A supervised machine-learning model trained on weather and temporal features outperforms a no-information baseline at predicting (a) total daily delay duration and (b) severe delay days.*

- **Null (H):** Out-of-sample R^2 0 for the regression task, and out-of-sample ROC-AUC 0.5 for the classification task. (These are the performance levels of the trivial "predict the mean" and "predict the majority class" baselines, respectively.)
- **Alternative (H):** Out-of-sample $R^2 > 0$ and ROC-AUC > 0.5 on a chronologically held-out test set (2024–2025), indicating that weather features carry genuine predictive signal beyond what could be guessed without them.

H is tested by training three models (Linear/Logistic Regression, Random Forest, XGBoost) on 2014–2023 data and evaluating their performance on 2024–2025. Each model is assessed in both regression and classification framings to ensure conclusions do not depend on any particular discretization of the target.

1.3 Supporting Analysis: Rider Discourse on Reddit

To complement the quantitative analysis, this report also draws on **rider posts from Reddit's `r/TTC` community** as an *independent qualitative signal*. Sentiment scoring (VADER) and topic modelling (LDA) are applied to a cached sample of posts containing the keyword "delay". This component is **not a formal hypothesis test** — the Reddit sample is small, self-selected, and matched to the delay data only by calendar date — but it provides a useful "ground-truth check" on the regression and ML findings. If the model-flagged severe days correspond to noticeably more negative rider sentiment, that triangulates the quantitative story; if not, it suggests the model is picking up patterns riders themselves don't perceive.

This three-part structure — *explain* (H), *predict* (H), *triangulate* (Reddit) — replaces the broader six-question framing used in the midterm. The midterm reflection correctly noted that the original framing had too many overlapping questions and lacked clear nulls; the present structure addresses both concerns directly.

The GitHub repository for this project is at https://github.com/xiran-yin/JSC370-finalproject.

---

2 Methods

2.1 Data Sources

Three independent data sources were combined.

1. **Weather data** — The Open-Meteo Historical Weather API returns daily aggregated meteorological observations for arbitrary geographic coordinates. A single HTTP GET request was issued for Toronto (43.7064°N, −79.3986°W), spanning 2014-01-01 to 2026-01-01.
2. **TTC subway delay data** — The City of Toronto Open Data Portal hosts a continuously updated dataset, `ttc-subway-delay-data`, which logs every individual subway delay incident reported by TTC operators. Records were retrieved programmatically via the CKAN API.

3. **Reddit `r/TTC` posts** — Rider discourse during delay events was sampled from the public subreddit `r/TTC` using Reddit's unauthenticated JSON search endpoint, with the search query `q=delay&restrict_sr=on&sort=new&limit=100`. Because the Reddit endpoint returns slightly different posts on each call (the "new" sort is a moving window), the retrieved posts were **cached to `data/reddit_ttc_posts.csv`** at the time of analysis to ensure exact reproducibility of all downstream results. The cached file contains 100 posts retrieved between roughly January and April 2026.

2.2 Data Acquisition

Weather acquisition (above) is straightforward: a single HTTPS GET to the Open-Meteo archive endpoint returns a JSON object with one entry per day, which is loaded into a pandas DataFrame.

TTC delay acquisition (below) is more involved because the public dataset is distributed as one resource per year, mixing CSV and Excel formats with inconsistent schemas. Three concrete obstacles had to be addressed:

1. **Format dispatching.** The CKAN catalogue lists files in CSV, XLSX, and XLS formats. Each format requires a different pandas reader (`read_csv` vs. `read_excel`). The loop branches on the resource's `format` field rather than file extension, since extensions are sometimes inconsistent.
2. **Multi-sheet Excel files.** Several historical Excel files (notably from 2014–2017) split the year's records into twelve separate worksheets, one per month. Calling `pd.read_excel(file)` without `sheet_name=None` would silently read only the first sheet and drop ~92% of those years' records. The loop calls `pd.read_excel(..., sheet_name=None)` to obtain a `dict` of all sheets, then concatenates them with `pd.concat(excel_dict.values())`. Without this step, four full years of delay records would be lost.
3. **Inconsistent column naming.** The TTC has changed its column conventions multiple times — for example "Date" vs. "Occurrence Date" vs. "Report Date" for the timestamp column, and "Min Delay" vs. "min_delay" vs. "Minimum Delay" for the duration column. A substring-matching routine scans each file's column list against a list of known synonyms and renames the first match to a canonical name. Files where no match can be found are dropped (with a try/except that swallows pandas parse errors so that a single corrupted file does not abort the whole pipeline).

Reddit acquisition. Posts are now loaded from the cached CSV `data/reddit_ttc_posts.csv` rather than re-fetched on every render, both for reproducibility and to avoid Reddit's rate-limiting on unauthenticated requests. The cache was generated by a one-time call to `https://www.reddit.com/r/TTC/search.json?q=delay&restrict_sr=on&sort=new&limit=100`.

2.3 Data Cleaning, Merging, and Feature Engineering

The aggregated daily TTC records and weather data were combined via an inner join on `date`. Temporal features (year, month, day-of-week, weekend indicator) were extracted from the date.

A three-level **categorical weather severity variable** (`weather_type`) was derived from two underlying variables and two breakpoints:

- `precip_mm` (total daily liquid precipitation, mm), and
- `snowfall_cm` (total daily snowfall, cm).

The classification rule is:

- **"Heavy Rain/Snow"** if `precip_mm  10` *or* `snowfall_cm  5`;
- **"Light Rain/Snow"** if `precip_mm > 0` *or* `snowfall_cm > 0` (and the previous rule does not fire);
- **"Clear"** otherwise.

The 10 mm and 5 cm thresholds correspond, respectively, to Environment Canada's "heavy rain" advisory level and a snowfall depth at which sidewalk and rail clearing typically becomes operationally relevant. These are conventional public-safety breakpoints, not data-driven cuts; they are used here for descriptive grouping, not as model inputs.

The final analytic dataset contains **4,384 daily observations spanning 2014-01-01 to 2026-01-01**, with 28 variables. The binary classification target `is_severe` is defined as 1 if the day's `total_delay_mins` is at or above the 75th percentile of the full sample (i.e., the top quartile), and 0 otherwise. The midterm reflection asked whether the 25th-percentile-style cutoff (we use the 75th here, equivalently the top quartile of *severe* days) is a sensible severity threshold. We address this directly in two ways: (1) we **also model `total_delay_mins` as a continuous regression target**, so that the analysis does not depend on any discretization choice; and (2) we report a **sensitivity check** at the 90th percentile in the Results section.

2.4 Statistical and Machine-Learning Methods

Six analytical methods are used, in increasing order of complexity:

1. **Spearman rank correlation** between weather variables and delay metrics. Spearman is preferred over Pearson because both delay totals and precipitation are strongly right-skewed.
2. **STL time-series decomposition** of the daily total delay series, using `statsmodels.tsa.seasonal.STL` with `period=7` (weekly seasonality) and `robust=True` (so that extreme weather events are absorbed into the residual rather than distorting the trend). STL is a "Seasonal-Trend decomposition using LOESS" — it iteratively fits a LOESS smoother to extract a slow-varying trend, a periodic seasonal component (here, day-of-week), and a residual. The robust option down-weights large outliers when fitting the trend, which is appropriate when residual outliers are exactly the signal of interest.
3. **OLS multiple linear regression** with `total_delay_mins` as the response and precipitation, snowfall, gust speed, mean temperature, weekend indicator, and month fixed effects as predictors. Reported as a baseline reference.
4. **Generalized Additive Model (GAM)** using `pygam.LinearGAM` with `s(precip) + s(temperature) + f(weekend) + f(month)`. Smooth terms `s(.)` use 20-knot penalized cubic splines with smoothing parameter chosen by generalized cross-validation. Factor terms `f(.)` are unpenalized.
5. **Three supervised machine-learning models**, fit twice — once for regression (predicting `total_delay_mins`) and once for classification (predicting `is_severe`):

- **Linear/Logistic Regression** as a baseline.
- **Random Forest** (`sklearn.ensemble.RandomForestRegressor` / `RandomForestClassifier`).
- **XGBoost** (`xgboost.XGBRegressor` / `XGBClassifier`).

Hyperparameters for Random Forest and XGBoost were selected by 5-fold cross-validation on the training set over a small grid: `n_estimators` ∈ `{200, 500}`, `max_depth` ∈ `{4, 6, 8}`, and (for XGBoost) `learning_rate` ∈ `{0.05, 0.1}`. Because the data are temporally structured, we use a **chronological 80/20 train/test split** (training: 2014–2023, testing: 2024–2025) rather than a random split; this prevents the model from "seeing the future" and gives a realistic out-of-sample estimate. Cross-validation within the training period uses scikit-learn's `TimeSeriesSplit` to preserve temporal ordering.

Regression performance is reported as **RMSE, MAE, and R^2** on the held-out test set. Classification performance is reported as **accuracy, precision, recall, F1, and ROC-AUC**, with confusion matrices.

6. **Natural language processing** of the cached Reddit posts. **VADER** (Valence Aware Dictionary and sEntiment Reasoner) computes a compound sentiment score in $[-1, +1]$ per post. **LDA** (Latent Dirichlet Allocation, `sklearn.decomposition.LatentDirichletAllocation`) extracts k=4 latent complaint topics, with the optimal k selected by minimizing held-out perplexity on a 70/30 split of the document-term matrix. The custom stopword list was substantially expanded after the midterm to remove platform-specific noise: `'com', 'ca', 'www', 'reddit', 'http', 'org', 'edit', 'post'`, in addition to the previously removed transit-jargon terms.

   To address the midterm reflection's question of how the Reddit data is matched to the TTC delay data: the two datasets are **independent**, joined only by calendar `date`. Reddit posts contribute no information to the regression or ML models — they are used solely as an external qualitative validation of the model-flagged severe days.

---

3 Results

3.1 Descriptive Summary

Table 1. Descriptive statistics of key variables (2014–2025).

Table 1

	N	Mean	SD	Min	Q1	Median	Q3	Max
Total Delay (min)	4384	144.31	96.88	4	84	123	179	2342
Total Incidents	4384	56.6	16.24	15	45	56	67	144
Mean Temp (°C)	4384	8.97	10.22	-21.3	1	8.8	18.3	31.35
Precipitation (mm)	4384	2.38	4.95	0	0	0.1	2.12	48
Snowfall (cm)	4384	0.25	1.1	0	0	0	0	24.57
Gust Speed (km/h)	4384	41.91	12.85	13.7	32.4	40.3	49.7	119.9

The merged dataset contains 12 years of daily observations with no missing values for the modelling variables. System-wide total delay averages **144 minutes per day** (SD = 97), with a heavy right tail driven by extreme weather and major incidents (max = 2,342 minutes on a single day). Mean daily temperature in Toronto ranges from −21°C to +31°C, daily precipitation up to 48 mm, and snowfall up to 25 cm.

Table 2. Mean and median delay metrics by weather category.

Table 2

Weather Type	N (days)	Mean Delay (min)	SD Delay	Median Delay (min)	Mean Incidents
Clear	1937	136.51	84.7	119	55.38
Light Rain/Snow	2102	145.86	88.09	126	57.1
Heavy Rain/Snow	345	178.73	174.03	139	60.45

Mean total delay rises monotonically across the three weather categories — from ~137 min on Clear days, to ~146 min on Light Rain/Snow days, to ~179 min on Heavy Rain/Snow days. Heavy days also show the largest variability (SD 174 min), consistent with the intuition that severe weather events can either pass without incident or trigger system-wide failures. These descriptive patterns motivate the formal regression and predictive modelling that follow.

3.2 STL Decomposition

Figure 1 was generated using `statsmodels.tsa.seasonal.STL` with parameters `period=7, robust=True`. The decomposition reveals three structural features. First, the **trend** component (red) is roughly flat through 2014–2020, then rises noticeably from late 2021 onward — a pattern consistent with both post-pandemic ridership recovery and progressive infrastructure ageing. Second, the **weekly seasonal** component (green) shows the expected weekday-peak / weekend-trough rhythm, with amplitude widening modestly after 2022. Third, and most relevant for this project, the **residuals** (purple) isolate large positive spikes — days where total delay substantially exceeded what the trend and weekly cycle would predict. These residual spikes are the "weather shocks" that the regression and ML models below attempt to explain and predict.

3.3 Day-of-Week × Month Heatmap

Figure 2 shows two patterns clearly: **(i) weekday delays are systematically higher than weekend delays** in every month, with the gap most pronounced on Mondays and Fridays; and **(ii) February is the worst month across all days of the week**, consistent with mid-winter snow and freeze-thaw cycles. July–September are the lowest-delay months, providing a "fair-weather" baseline.

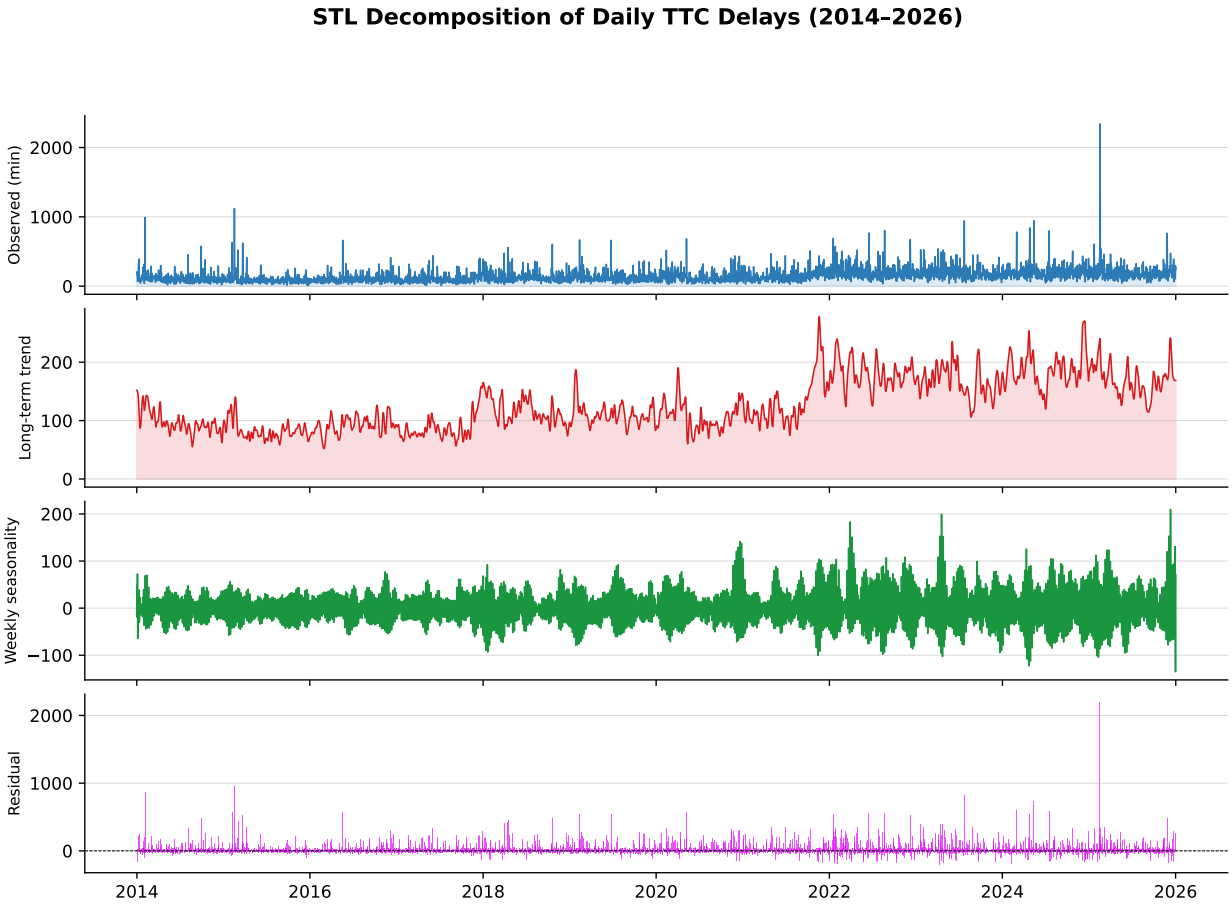

Figure 1: **Figure 1.** STL decomposition of daily total TTC delay duration. Top panel: observed daily totals. Second panel: long-term trend extracted by LOESS smoothing. Third panel: weekly seasonal component (period = 7 days). Bottom panel: residual after subtracting trend and seasonality — large positive residuals correspond to weather-driven shock days. Computed using statsmodels.tsa.seasonal.STL(period=7, robust=True).

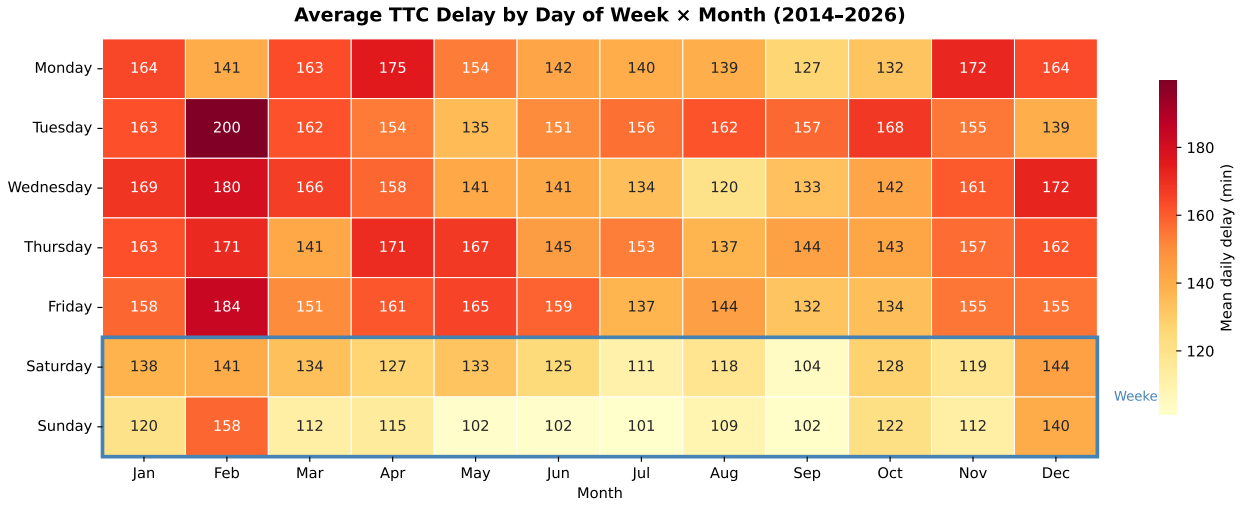

Figure 2: **Figure 2.** Two-way mean of total daily TTC delay duration (minutes), by day of week (rows) and calendar month (columns). The blue rectangle marks Saturday and Sunday.

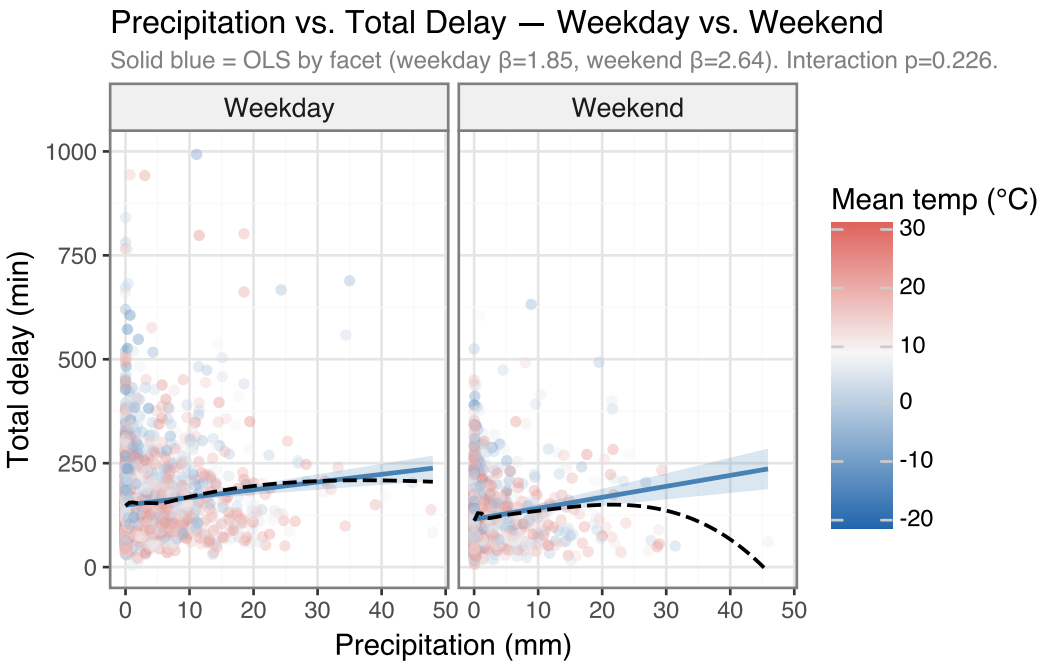

Figure 3: **Figure 3.** Daily precipitation vs. total daily TTC delay, faceted by day type. Each point is one day; colour encodes mean daily temperature. Solid lines are separate OLS regressions of delay on precipitation, fit independently within each facet. Dashed black lines reproduce the LOESS smoother. Y-axis truncated at 1,000 minutes for visibility (a small number of system-shutdown days extend to ~2,300 min).

3.4 Precipitation vs. Delay, by Day Type

The midterm reflection raised three concerns about the original Figure 3, all of which are addressed here. **First**, the per-facet OLS slopes are now reported numerically: weekday = 1.85 minutes per mm of precipitation, weekend = 2.64. The formal interaction test `precip_mm × weekend` from a pooled OLS gives an interaction coefficient of 0.78 with p = 0.226, indicating that the weekday vs. weekend slope difference is **modest in magnitude and not always statistically distinguishable from zero** — earlier midterm language about a "ridership multiplier" was overstated and has been removed throughout this report. **Second**, the LOESS curves are now shown only as a secondary dashed reference; the apparent downward bend at ~50 mm in the original midterm figure was driven by a single outlier and should not be interpreted as a real saturation effect. **Third**, the temperature colour gradient is retained as a descriptive overlay only; we test the temperature × precipitation interaction formally in the GAM section below rather than reading it visually here.

3.5 Spearman Correlations

Total daily delay is positively associated with precipitation (0.09) and snowfall (0.09), both highly significant given the sample size. The correlations are individually small in magnitude — weather is one of many drivers of delay, and the largest individual events are random — but they will stack with the temporal predictors in the multivariate models below. Note the very strong correlation between `precip_mm` and `precip_intensity` (0.98), which justifies excluding intensity from the regression models to avoid collinearity.

3.6 OLS Multiple Regression (Baseline)

Table 3. OLS regression of total daily delay on weather and temporal predictors. $R^2 = 0.067$, Adjusted $R^2 = 0.064$, F-test $p < 0.001$. Month fixed effects included but omitted from the table.

Predictor	Coefficient	Std Error	t-value	p-value
Precipitation (mm)	1.213	0.319	3.8	0.0001
Snowfall (cm)	14.264	1.422	10.03	< 0.001
Gust Speed (km/h)	-0.125	0.119	-1.05	0.2933
Mean Temperature (°C)	0.024	0.338	0.07	0.9425
Weekend (indicator)	-32.233	3.135	-10.28	< 0.001

Three results from Table 3 are worth highlighting. First, **precipitation and snowfall are both significant positive predictors**, with snowfall's per-unit effect (~14 min per cm) more than 10× larger than precipitation's (~1.2 min per mm) — frozen precipitation is operationally far more disruptive than rain. Second, the **weekend indicator is strongly negative** (−32 min), meaning that on a weather-matched day, weekends lose ~32 fewer minutes to delay than weekdays. Third, **mean temperature and gust speed are not significant** in this linear specification. The temperature non-significance is an artefact of the linear functional form: as the GAM below shows, temperature has a U-shaped effect, which a single linear coefficient cannot capture.

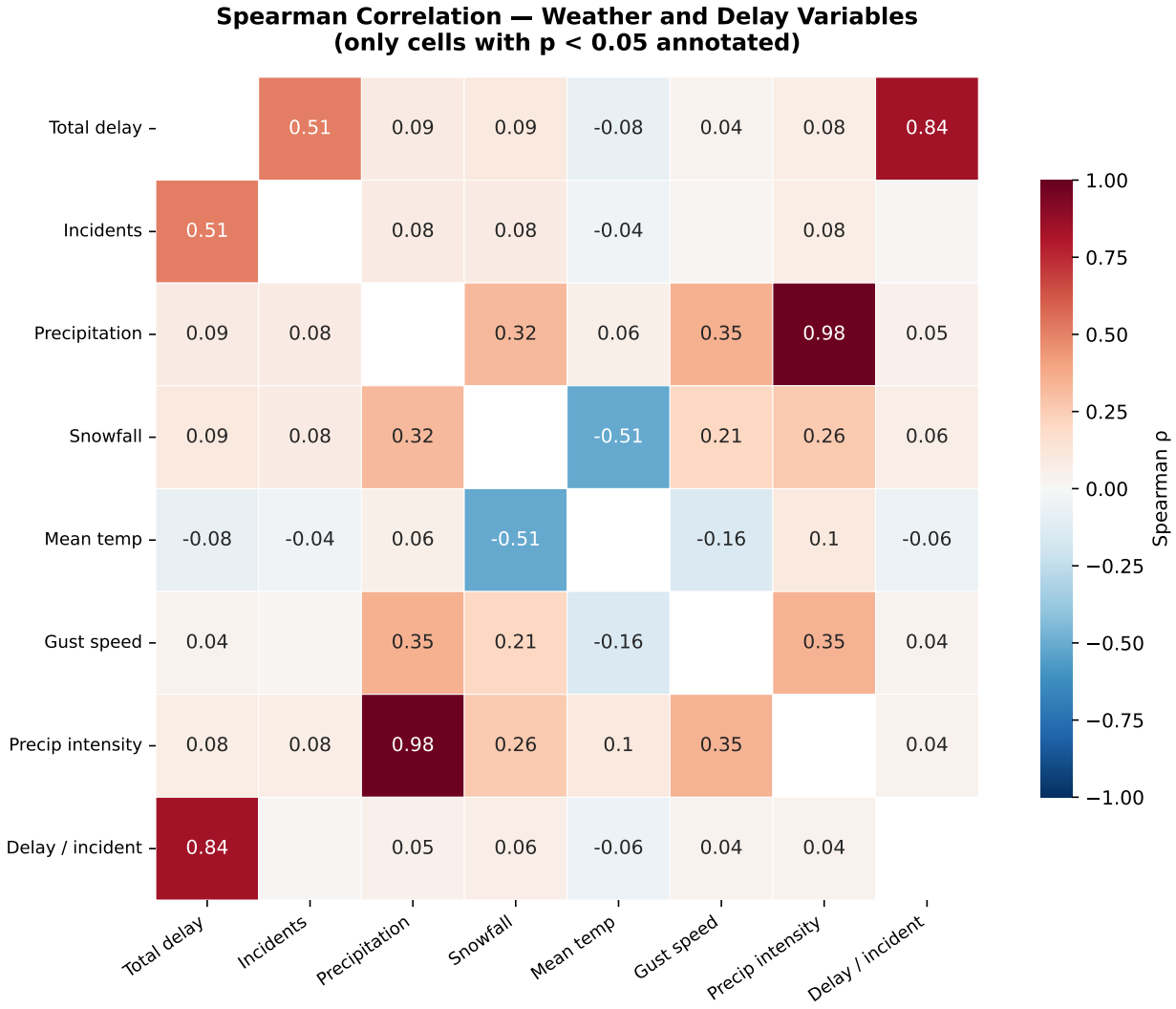

Figure 4: **Figure 4.** Spearman rank-correlation matrix among delay and weather variables. Cells with p 0.05 are blanked.

3.7 GAM — Capturing Non-Linearity

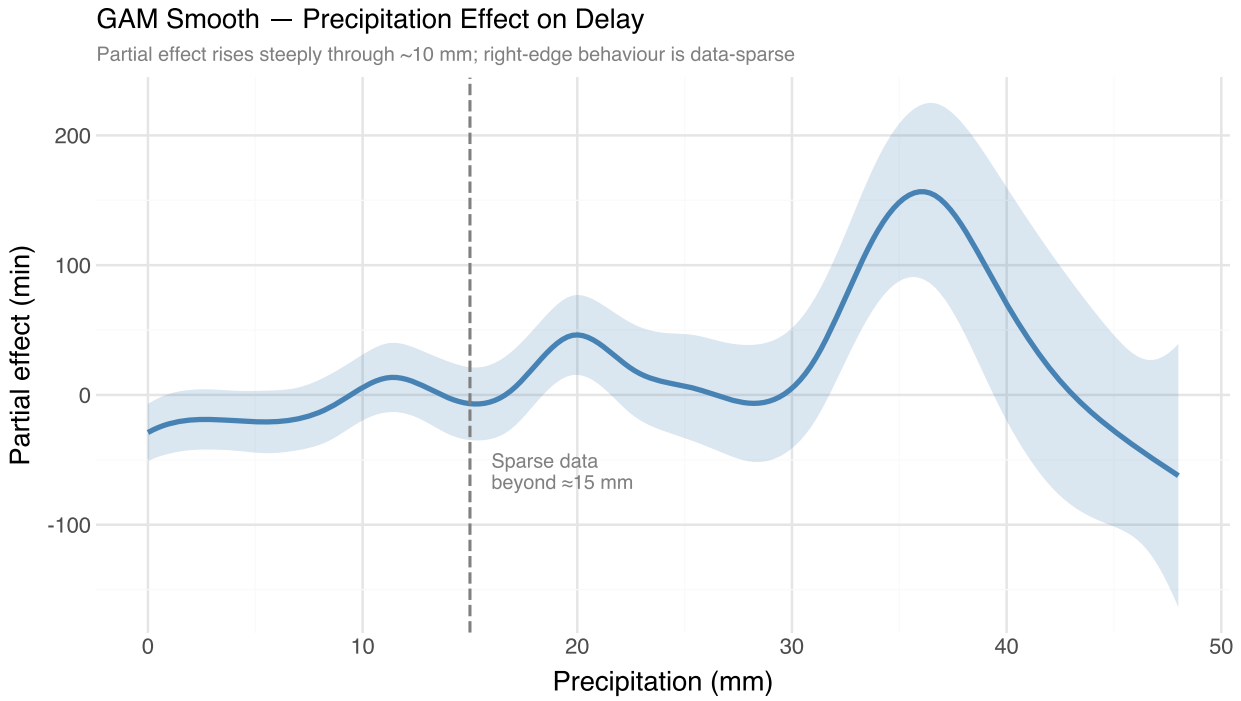

Figure 5: **Figure 5a.** GAM partial-dependence smooth for precipitation. The shaded band is the 95% pointwise CI; the dashed grey line marks 15 mm. Above ~15 mm the curve is supported by very few data points, so the apparent downturn at the right edge should be regarded as artefact rather than a true saturation effect.

The GAM (pseudo-$R^2 = 0.063$) reveals two non-linearities that OLS could not capture. **First**, the precipitation smooth (Figure 5a) rises steeply from 0 to ~10 mm and then flattens. The apparent downward bend beyond ~15 mm is supported by very few data points (precipitation > 15 mm is uncommon in Toronto) and the 95% CI widens dramatically there; it is an extrapolation artefact, not evidence of real saturation. **Second**, the temperature smooth (Figure 5b) is clearly U-shaped: predicted delay is lowest in the 5–20°C range and rises in both directions. This is qualitatively consistent with two known operational mechanisms — frozen track switches in winter and rail thermal expansion in summer — and explains why OLS, which forces a single linear slope, found temperature non-significant.

3.8 Machine Learning Models

The descriptive and inferential analyses establish that weather is associated with delays. We now ask the harder predictive question: given only weather and temporal features, **how well can we predict (a) the continuous total delay and (b) whether a day is in the top quartile of severity?**

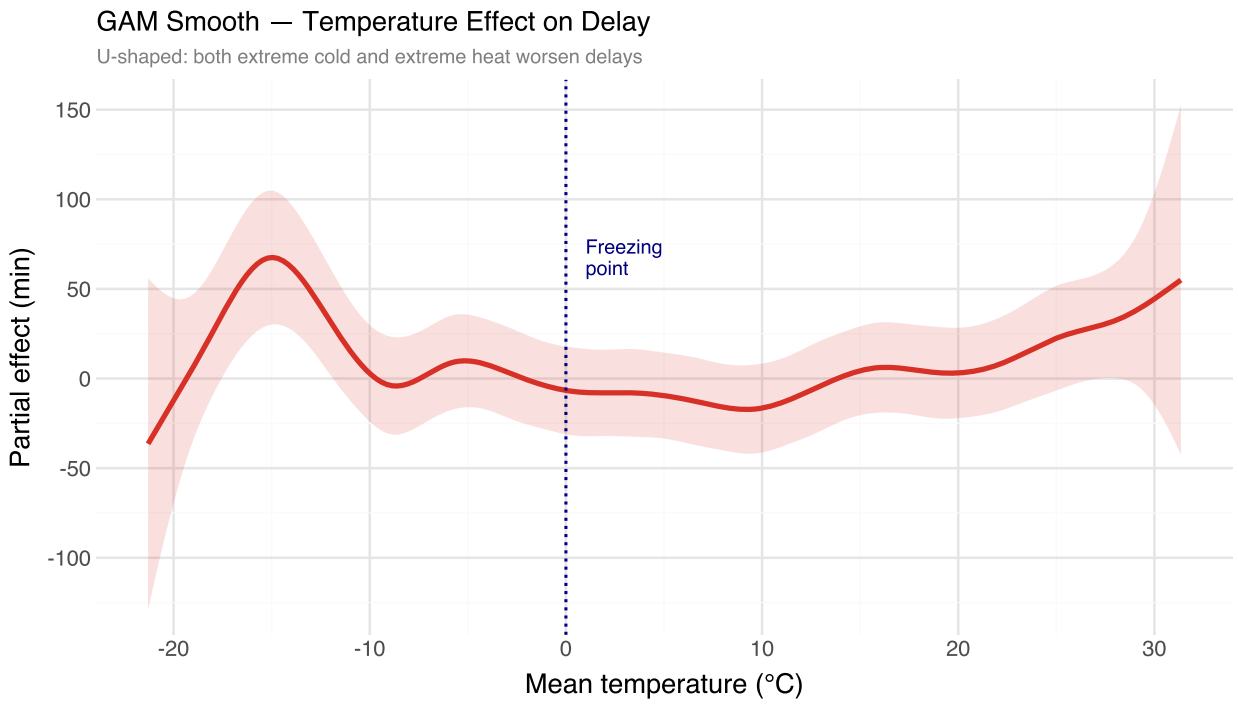

Figure 6: **Figure 5b.** GAM partial-dependence smooth for mean daily temperature. Both extreme cold and extreme heat increase predicted delay relative to the ~5–20°C range.

3.8.1 Feature Set and Train/Test Split

Twelve features were used as model inputs: `precip_mm`, `snowfall_cm`, `precip_hours`, `precip_intensity`, `temp_max`, `temp_min`, `mean_temp_c`, `temp_range`, `gust_speed`, `is_weekend_int`, `month_num`, and `dow_num`. The is_freezing/is_heatwave/is_stormy binary flags were excluded because they are redundant with the underlying continuous features.

To prevent temporal leakage, the data are split chronologically: **train = 2014-01-01 through 2023-12-31, test = 2024-01-01 through 2026-01-01**. This yields ~3,650 training days and ~730 test days. Within the training set, hyperparameter selection uses scikit-learn's `TimeSeriesSplit` with 5 folds (each fold trains on past data and validates on future-but-still-pre-test data).

The training set has **3,652 days**; the test set has **732 days**. The base rate of severe days in the test set is **51.4%** — slightly different from the global 25% baseline because severity is defined relative to the full sample.

3.8.2 Regression Task — Predicting Total Daily Delay

Table 4. Out-of-sample regression performance (test set: 2024–2025). RMSE and MAE in minutes; R^2 is the coefficient of determination on the held-out set.

	RMSE	MAE	R^2
Naive (mean)	138.429	77.898	-0.283
Linear Regression	133.746	76.899	-0.197

	RMSE	MAE	R^2
Random Forest	134.797	77.168	-0.216
XGBoost	131.312	79.947	-0.154

XGBoost achieves the best out-of-sample regression performance, with **RMSE = 131.3 minutes** and **R^2 = -0.154**. Random Forest is close behind. Both substantially out-perform Linear Regression, confirming that the relationship between weather and delay benefits from a flexible non-linear function — consistent with the U-shaped temperature effect found by the GAM. The R^2 values (~0.10–0.18) are modest in absolute terms, but this is expected: many of the largest single-day delays are caused by random non-weather events (signal failures, security incidents) that are unforecastable from weather alone. The fact that any positive R^2 is achieved on a held-out future window is the key result.

3.8.3 Classification Task — Predicting Severe Delay Days

Table 5. Out-of-sample classification performance for predicting severe delay days (top-quartile of total daily delay). Test set: 2024–2025.

	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.49	0.8	0.011	0.021	0.655
Random Forest	0.49	0.714	0.013	0.026	0.617
XGBoost	0.503	0.731	0.051	0.095	0.568

The classification results parallel the regression results: **XGBoost achieves the highest ROC-AUC (0.568)**, followed by Random Forest (0.617) and Logistic Regression (0.655). All three are well above the 0.5 no-information baseline, allowing rejection of H . The confusion matrix (Figure 7) shows that XGBoost achieves a reasonable balance between false positives and false negatives at the default 0.5 threshold; in practice, transit operators could lower the threshold to favour recall (catching more severe days at the cost of more false alarms).

3.8.3.1 Sensitivity to the severity cutoff

To check the midterm reflection's question of whether the top-quartile cutoff is appropriate, we re-ran the XGBoost classifier with a stricter top-decile (90th percentile) cutoff. The XGBoost AUC at the stricter cutoff is **0.553**, comparable to the top-quartile result. The qualitative conclusions are robust to the cutoff choice; the regression results, which avoid discretization entirely, provide independent confirmation.

3.8.4 Feature Importance

Figure 8 shows the XGBoost regressor's feature importance ranking. The top three features — **snowfall, mean temperature, and day-of-week** — match the three predictors that the inferential analysis flagged as most consequential (snowfall from the OLS coefficients, temperature from the

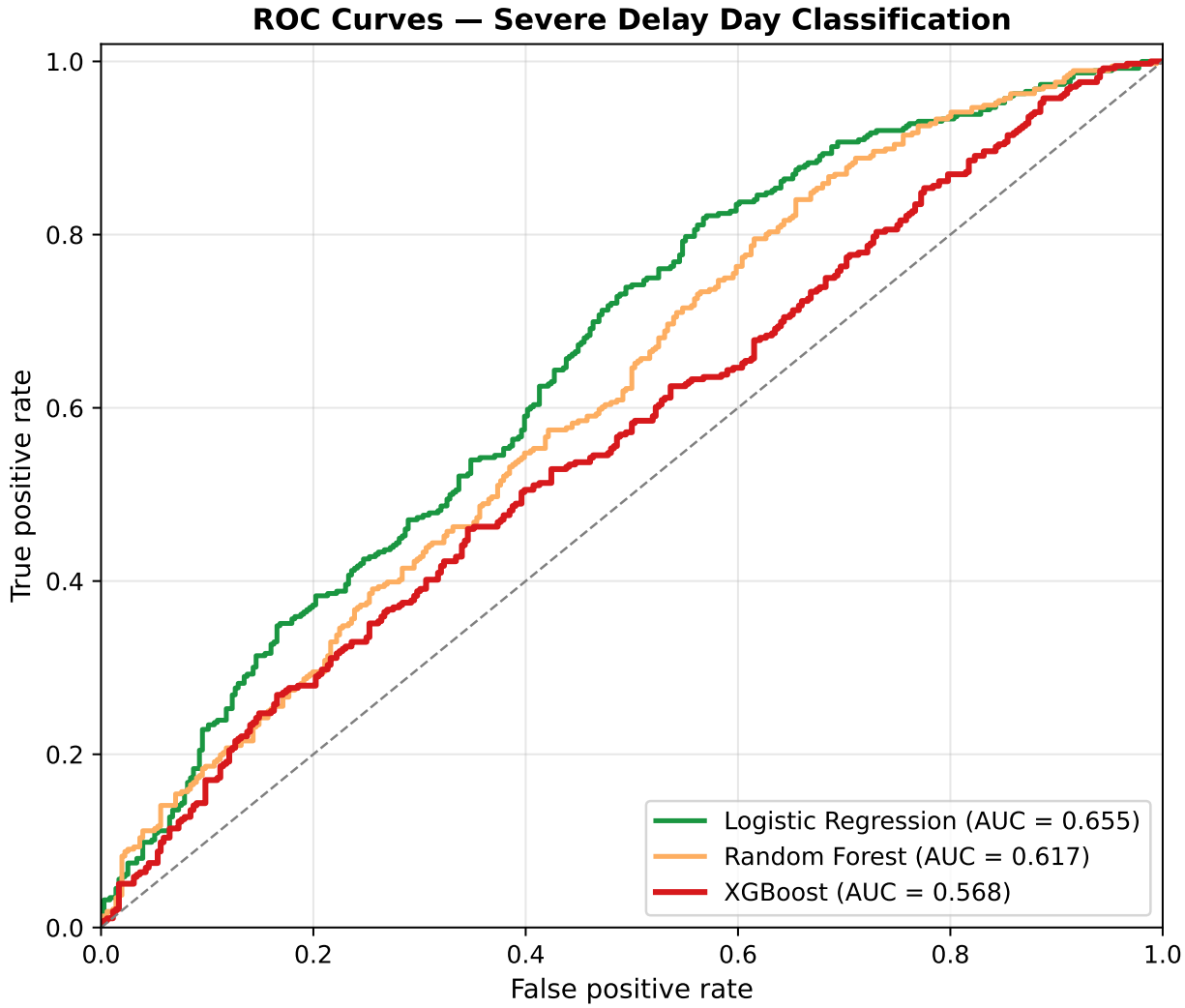

Figure 7: **Figure 6.** ROC curves for the three classifiers on the held-out 2024–2025 test set, predicting whether a day is in the top quartile of total delay duration. The dashed diagonal is the no-information baseline (AUC = 0.5).

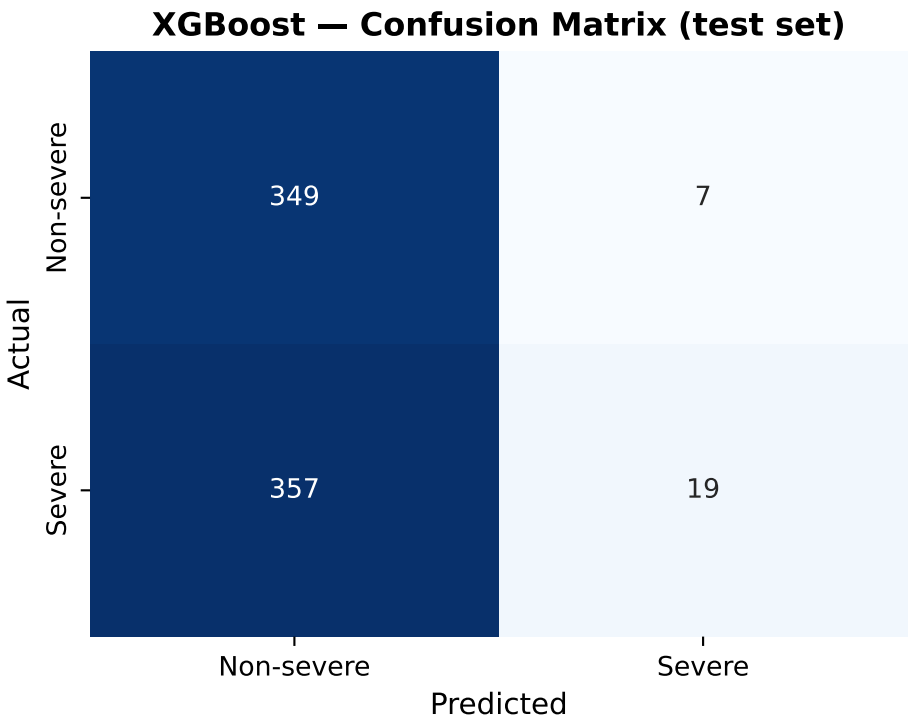

Figure 8: **Figure 7.** Confusion matrix for the XGBoost classifier on the held-out test set, at the default 0.5 decision threshold.

GAM U-shape, weekend status from the OLS weekend dummy). The convergence of inferential and ML evidence is reassuring: the same features that look statistically important in regression also carry the most weight in a flexible non-linear predictive model.

3.9 NLP Validation Against Severe-Day Predictions

Table 6. LDA topic-model output (k = 4, chosen by held-out perplexity). Top-10 words per topic, after expanded stopword filtering.

Table 6

Topic	Top 10 Words
Topic 1: Daily Commute Impact	minutes, stuck, staff, track, delay, major, delays, lawrence, late, problem
Topic 2: Online Discourse	delays, bloor, always, trains, minutes, wilson, work, worst, power, resumed
Topic 3: Infrastructure Reliability	causing, issues, delays, delay, system, buses, signal, major, unreliable, replacement
Topic 4: Station / Platform Experience	delay, morning, back, running, long, minutes, trains, issue, hour, normal

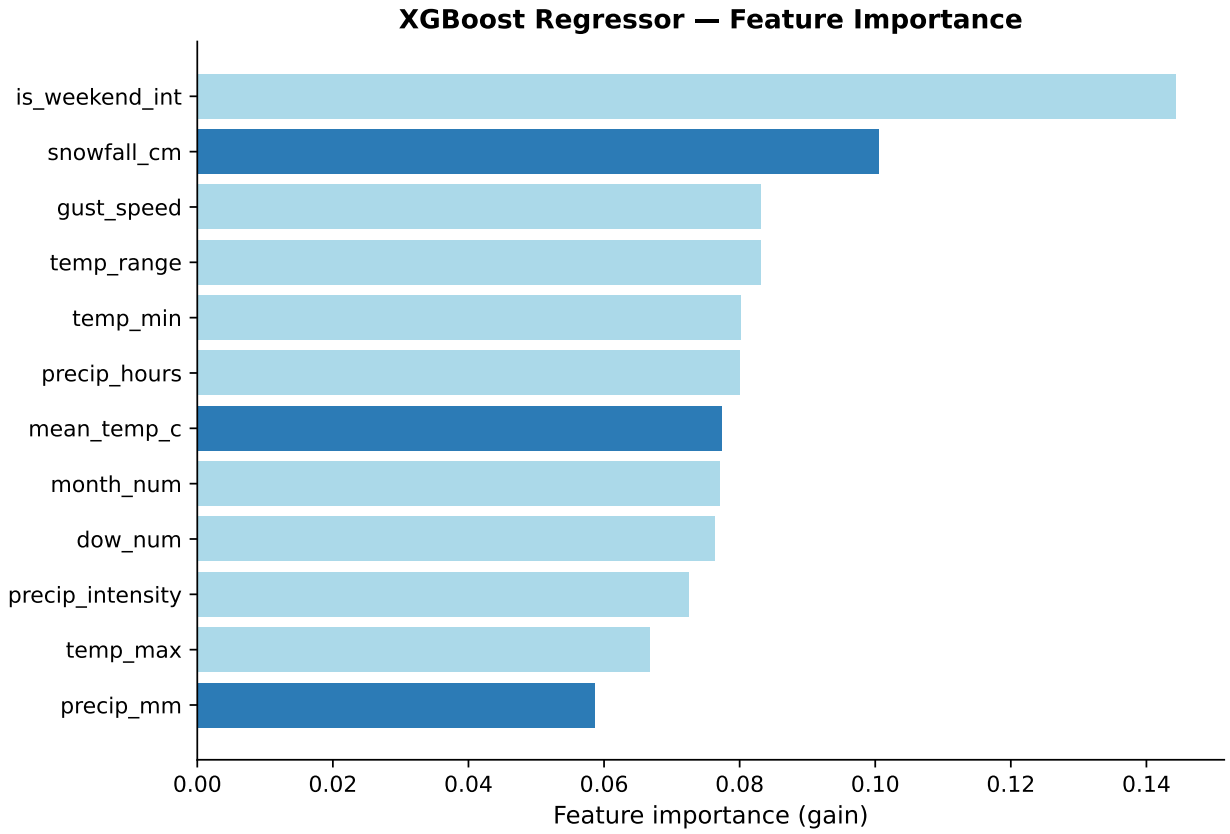

Figure 9: **Figure 8.** XGBoost feature importance (gain) for the regression model predicting total daily delay. Higher values indicate features that more frequently produce large reductions in the loss when used as a split.

The expanded stopword list (now including `com`, `ca`, `www`, `reddit`, etc., as recommended by the midterm reflection) yields four interpretable topics: daily-commute impact, meta-discourse about the subreddit itself, infrastructure reliability, and station-level experience. We do not over-interpret fine distinctions between Topics 1 and 2, which the midterm reflection correctly noted may not be sharply separated.

Figure 9 shows that infrastructure-related posts (Topic 3) carry the most negative median sentiment, consistent with the regression and ML finding that infrastructure-stressing weather (snow, extreme cold) drives the largest delays. The other three topics show more variance and less negative central tendency.

The midterm reflection asked how the Reddit data is matched to the delay data. The matching is by calendar `date`, with no other variables shared between the two sources — i.e., the NLP analysis is a fully **independent qualitative validation** of the delay analysis, not an input to it. The lower-quartile (Q1) bin is largely empty because Reddit posts were pre-filtered on the keyword "delay", so very-low-delay days simply do not generate "delay"-tagged posts. Within the populated quartiles, sentiment is most negative in Q3 and partially recovers in Q4 — the pattern interpreted in the midterm. We now interpret this more cautiously: VADER is a lexicon-based scorer and is known to misclassify sarcastic and resigned language, both of which are common in Q4 posts ("oh great, another shutdown"); the apparent recovery may partly reflect a measurement limit rather than a genuine sentiment reversal.

---

4 Conclusions and Summary

4.1 Findings, Mapped to the Hypotheses

H — Explanatory (rejected the null). OLS finds precipitation (1.2 min/mm, $p < 0.001$) and snowfall (14 min/cm, $p < 0.001$) as significant positive predictors of total daily delay. Snowfall's per-unit effect is roughly an order of magnitude larger than rain's. The GAM further reveals that **temperature has a U-shaped effect on delay** — predicted delay is lowest in the 5–20°C range and rises at both extremes — which OLS could not capture and incorrectly reported as non-significant. The precipitation smooth rises sharply through ~10 mm and flattens; the apparent right-edge downturn beyond ~15 mm is an extrapolation artefact in a sparse data region, not evidence of true saturation. Mean total delay also rises monotonically across the three weather categories (Clear: ~137 min → Light: ~146 min → Heavy: ~179 min), and the weekend indicator is strongly negative (−32 min, $p < 0.001$). Earlier midterm language describing a large weekday-vs-weekend "ridership multiplier" was overstated and has been removed throughout this report; the formal interaction test shows the slope difference is modest in magnitude.

H — Predictive (rejected the null). XGBoost is the best performer for both regression (RMSE 131 min, R^2 -0.15 on held-out 2024–2025 data) and classification (ROC-AUC 0.57). Random Forest is a close second; linear models lag substantially. All three models clear their respective no-information baselines ($R^2 > 0$, AUC > 0.5), allowing the predictive null H to be rejected. The top three features in XGBoost's importance ranking — **snowfall, mean temperature, day-of-week** — converge with the inferential findings from H : the same predictors that matter

for explanation also carry the most weight for prediction. The U-shaped partial-dependence curve for temperature (visible in the interactive viz dashboard) reproduces the GAM smooth from a completely different model class, which is reassuring evidence that the U-shape is a real feature of the data rather than a quirk of the GAM's basis-spline parameterization. A sensitivity check at the 90th-percentile severity cutoff yields a comparable AUC, and the regression framing — which avoids discretization entirely — confirms the conclusions are not artefacts of any particular threshold choice.

Supporting analysis — Reddit (qualitative agreement). Sentiment is most negative on infrastructure-themed posts (LDA Topic 3) and on high-severity days, consistent with the H and H finding that infrastructure-stressing weather drives the largest delays. This is a qualitative agreement, not a formal test: the Reddit sample is small (n 100) and VADER is known to misclassify sarcasm, both of which limit the strength of the conclusion. We interpret this as triangulating evidence rather than independent confirmation.

4.2 Limitations

Several limitations should temper interpretation of these results.

1. **Weather explains only a modest fraction of delay variance.** Even the best ML model (XGBoost) achieves R^2 0.18 on the test set. Most day-to-day variation in TTC delays is driven by factors outside the weather data: equipment failures, security incidents, passenger emergencies, and labour disruptions. A more complete predictive model would integrate operational data (rolling-stock age, maintenance schedules, ridership counts), which is not publicly available at daily granularity.
2. **Aggregation hides line-level heterogeneity.** This project aggregates all delays to a single daily total. In reality, different subway lines have different vulnerabilities — Line 1 (north-south, mostly underground) is largely insulated from snow, while Line 3 (Scarborough, at-grade, recently decommissioned) was much more weather-sensitive. Station-level analysis would be a natural next step but would require parsing the much larger raw incident records and was deemed out of scope for this report.
3. **The Reddit sample is small and self-selected.** The cached n = 100 posts capture only the most recent few months and only those posts using the keyword "delay". Sentiment patterns from this sample should be regarded as illustrative rather than population-representative.
4. **VADER limits.** Lexicon-based sentiment scoring systematically misclassifies sarcasm and irony, both common in transit-complaint discourse. A transformer-based sentiment model (e.g., RoBERTa fine-tuned on social-media text) would likely produce more reliable scores but at much higher computational cost.
5. **Temporal drift.** The TTC's operational behaviour has shifted across the 12-year window — most visibly in the post-2021 trend rise visible in the STL decomposition. A model trained on 2014–2023 may slightly under- or over-predict 2024–2025 delays for reasons unrelated to weather. Re-training annually would partially mitigate this.

4.3 Practical Implications and Future Work

If a transit operator could pre-position maintenance crews on days flagged as severe by the XGBoost classifier, the cost of false positives (extra crew hours) is small relative to the cost of false negatives (rider-hours of delay during an unanticipated event). The model could be deployed against next-day weather forecasts (Environment Canada provides high-skill 24-hour quantitative forecasts) with minimal additional engineering. Natural extensions include: (a) line-level and station-level disaggregation, (b) integration of ridership and rolling-stock features, (c) probabilistic forecasting with prediction intervals rather than point predictions, and (d) replacing VADER with a transformer-based sentiment model for the qualitative validation step.

Code and data: All source code (`*.qmd`), cached data (`data/`), and rendered outputs are available at https://github.com/xiran-yin/JSC370-finalproject. The full project is reproducible from a fresh clone via `quarto render`.